Vision-based Simultaneous Localization And Mapping

Dr. Xiang Gao

Ph.D of Tsinghua University, Postdoc at Technical University of Munich
idriverplus.com

What we will cover in this course?

- Lecture 1. Introduction to 3D Geometry
- Lecture 2. Visual Odometry and Backend Optimization

What we will cover in this course?

- Lecture 1. Preliminary Knowledge
- Lecture 2. Optimization on Manifold
- Lecture 3. Visual Odometry
- Lecture 4. Backend Optimization

Contents

- Feature extraction and matching
- Optical Flow
- Pose estimation approaches
- Direct method

Contents

- Feature extraction and matching
- Optical Flow
- Pose estimation approaches
- Direct method

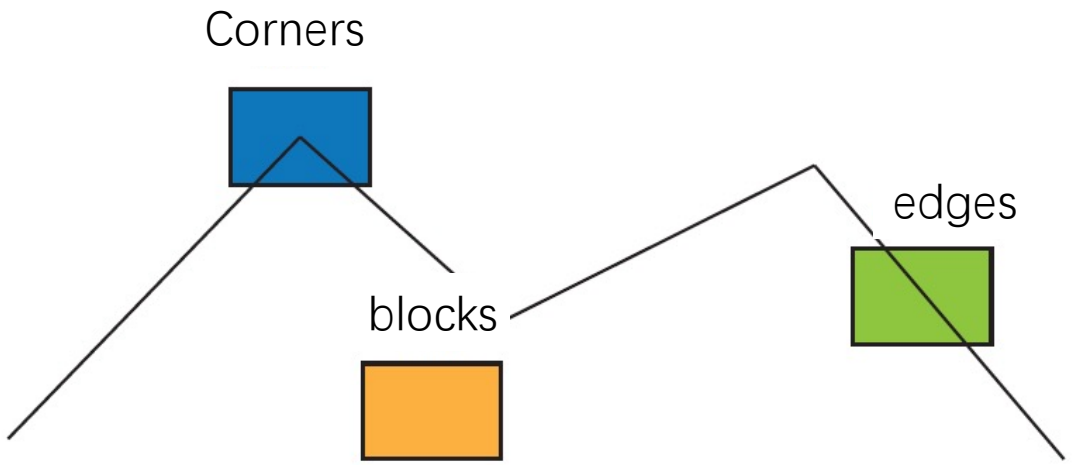
1. Feature Extraction and Matching

- Visual odometry steps
  - Find the corresponding points in the images
  - Estimate camera motion
  - Expand the map if needed

1. Feature Extraction and Matching

- We estimation camera pose by the observed landmarks
  - Landmarks: fixed in 3D space and can be observed in the image
  - Distinctive: landmarks should be easy to distinguish
- Image features are used as landmarks in visual SLAM

- Image features
  - Repeatable
  - Distinctive
  - Efficient
  - Local



1. Feature Extraction and Matching

- Feature
  - Keypoint: position, size, angle, score, etc.
  - Descriptor: encode the surrounding image information

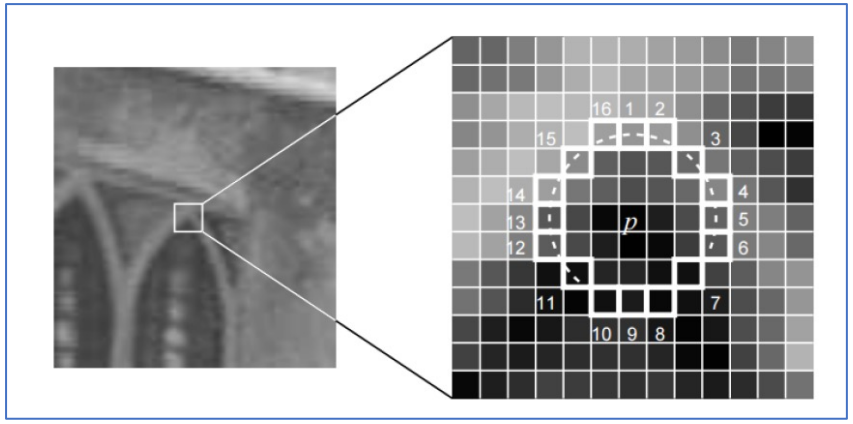
- Examples:
  - SIFT
  - SURF
  - ORB
  - etc (see OpenCV’ s features2d module)

features2d. 2D Features Framev

- Feature Detection and Description
  - FAST
  - MSER
  - MSER::MSER
  - MSER::operator()
  - ORB
  - ORB::ORB
  - ORB::operator()
  - BRISK
  - BRISK::BRISK
  - BRISK::BRISK
  - BRISK::operator()
  - FREAK
  - FREAK::FREAK
  - FREAK::selectPairs
- Common Interfaces of Feature Detectors
  - KeyPoint
  - KeyPoint::KeyPoint
  - FeatureDetector
  - FeatureDetector::detect
  - FeatureDetector::create
  - FastFeatureDetector
  - GoodFeaturesToTrackDetector
  - MserFeatureDetector
  - StarFeatureDetector
  - DenseFeatureDetector
  - SimpleBlobDetector
  - GridAdaptedFeatureDetector
  - PyramidAdaptedFeatureDetector
  - DynamicAdaptedFeatureDetector
  - DynamicAdaptedFeatureDetector::DynamicAdap

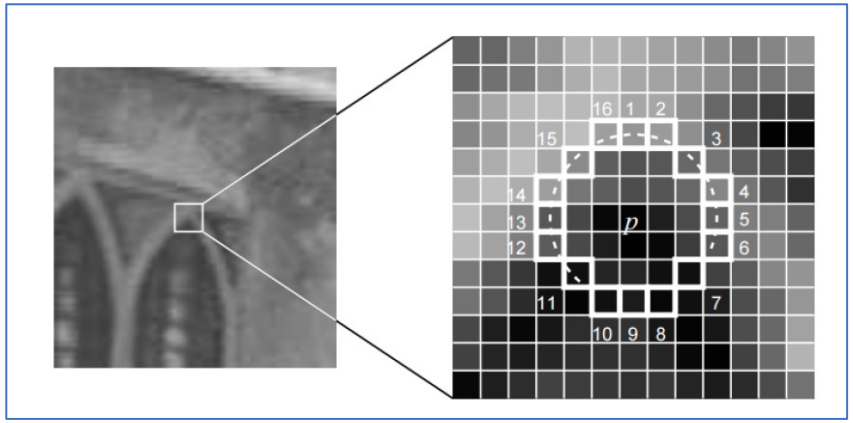
1. Feature Extraction and Matching

- Take ORB as an example
- ORB
  - Keypoint: Oriented FAST
  - Descriptor: Steer BRIEF



- FAST keypoint
  - P is a corner if we have continuous n points whose image intensity is larger/smaller than p over a threshold
  - Called FAST-n, e.g., FAST-12, FAST-10, FAST-9 …

1. Feature Extraction and Matching



- Oriented FAST
  - Compute an angle in the FAST
- In a image patch B, define its moment as:

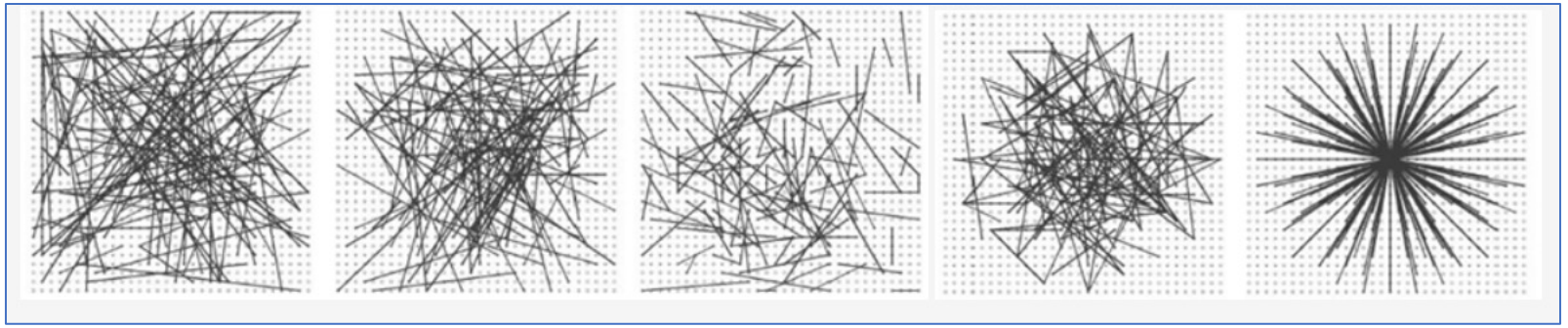
$$m_{pq} = \sum_{x,y \in B} x^p y^q I(x,y), \{p,q\} \in \{0,1\}$$

- Find the centroid of the patch: $\left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}}\right)$

- And compute the angle: $\theta = \arctan(m_{01}/m_{10})$

1. Feature Extraction and Matching

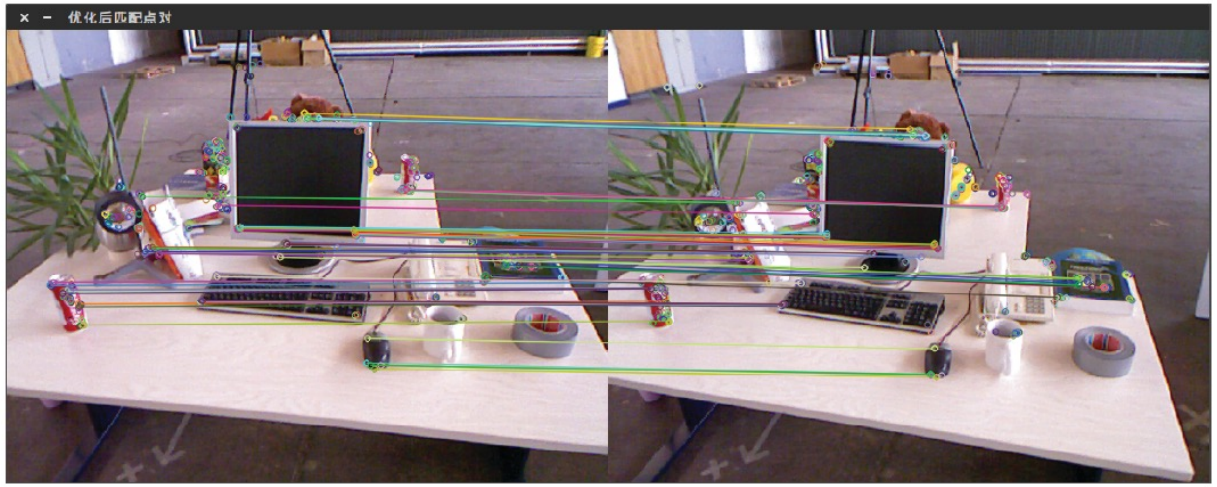
- BRIEF descriptor
  - Binary Robust Independent Elementary Features (BRIEF)
  - BRIEF-n: Compare n pairs of pixels around the keypoint
    - The pairs are chosen randomly or with a certain pattern



- ORB use steer (rotated) BRIEF
  - Rotate the BRIEF pattern according to the precomputed angle

1. Feature Extraction and Matching

- Feature matching
  - Compute the data association according the descriptor distance
  - Brute-force matching: compare each pairs of descriptors
  - FLANN: Fast approximate nearest neighbor
  - For binary descriptors like BRIEF and ORB, use Hamming distance:
    - Hamming(x,y) = number of different coefficients

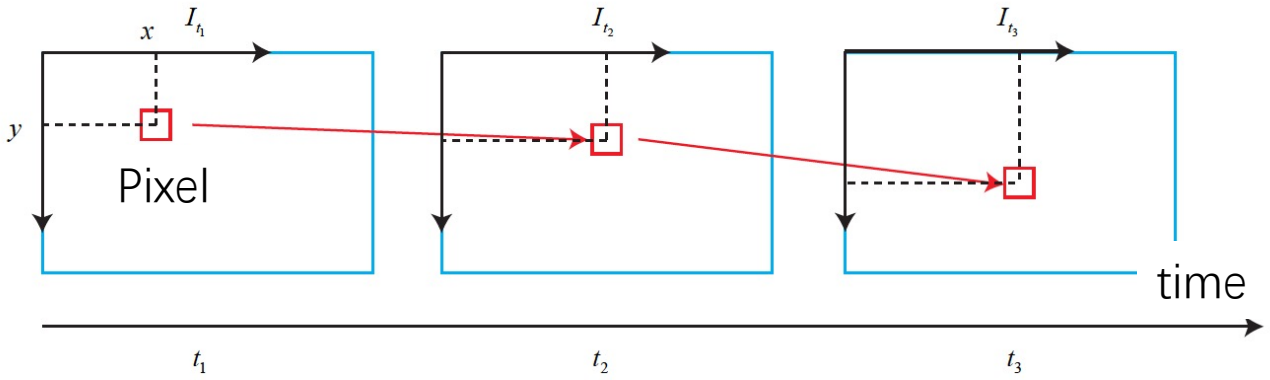


Contents

- Feature extraction and matching
- Optical Flow
- Pose estimation approaches
- Direct method

2. Optical Flow

- Optical Flow
  - Estimate the motion of pixels in continuous images
  - Sparse vs Dense Flow
    - Sparse: Lucas-Kanade (LK) flow
    - Dense: Horn-Schunck (HS) flow
  - Can be used to find corresponding pixels in images



2. Optical Flow

- How to estimate optical flow?
- Assume at time t we have a pixel at x,y, then its intensity is: $\boldsymbol{I}(x, y, t).$
- At t+dt, it moves to x+dx, y+dy, and the intensity $\boldsymbol{I}(x + \mathrm{d}x, y + \mathrm{d}y, t + \mathrm{d}t)$
- Brightness constancy assumption:

$$\boldsymbol{I}(x + \mathrm{d}x, y + \mathrm{d}y, t + \mathrm{d}t) = \boldsymbol{I}(x, y, t).$$

- Note this is really a strong and ideal assumption since brightness can be changed by highlight/shadow/occlusion/material/exposure and will not hold any more

2. Optical Flow

- With brightness constancy, we expand the assumption:

$$\boldsymbol{I}(x+\mathrm{d}x, y+\mathrm{d}y, t+\mathrm{d}t) = \boldsymbol{I}(x, y, t).$$

$$\boldsymbol{I}\left(x+\mathrm{d}x, y+\mathrm{d}y, t+\mathrm{d}t\right) \approx \boldsymbol{I}\left(x, y, t\right) + \frac{\partial \boldsymbol{I}}{\partial x}\mathrm{d}x + \frac{\partial \boldsymbol{I}}{\partial y}\mathrm{d}y + \frac{\partial \boldsymbol{I}}{\partial t}\mathrm{d}t.$$

- And obtain:

Gradient with time

$$\frac{\partial \boldsymbol{I}}{\partial x}\mathrm{d}x + \frac{\partial \boldsymbol{I}}{\partial y}\mathrm{d}y + \frac{\partial \boldsymbol{I}}{\partial t}\mathrm{d}t = 0. \qquad \frac{\partial \boldsymbol{I}}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial \boldsymbol{I}}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t} = -\frac{\partial \boldsymbol{I}}{\partial t}.$$

Gradient on x-axis

Gradient on y-axis

- Our object: compute dx/dt and dy/dt

2. Optical Flow

$$\frac{\partial \boldsymbol{I}}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t}+\frac{\partial \boldsymbol{I}}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t}=-\frac{\partial \boldsymbol{I}}{\partial t}.$$

- However it is a underdetermined linear equation
  - 2 unknowns and 1 equation
  - We need extra constraints: assume brightness constancy in a small window of $w \times w$ patch

$$\begin{bmatrix} \boldsymbol{I}_x & \boldsymbol{I}_y \end{bmatrix}_k \begin{bmatrix} u \\ v \end{bmatrix} = -\boldsymbol{I}_{tk}, \quad k = 1, \ldots, w^2.$$

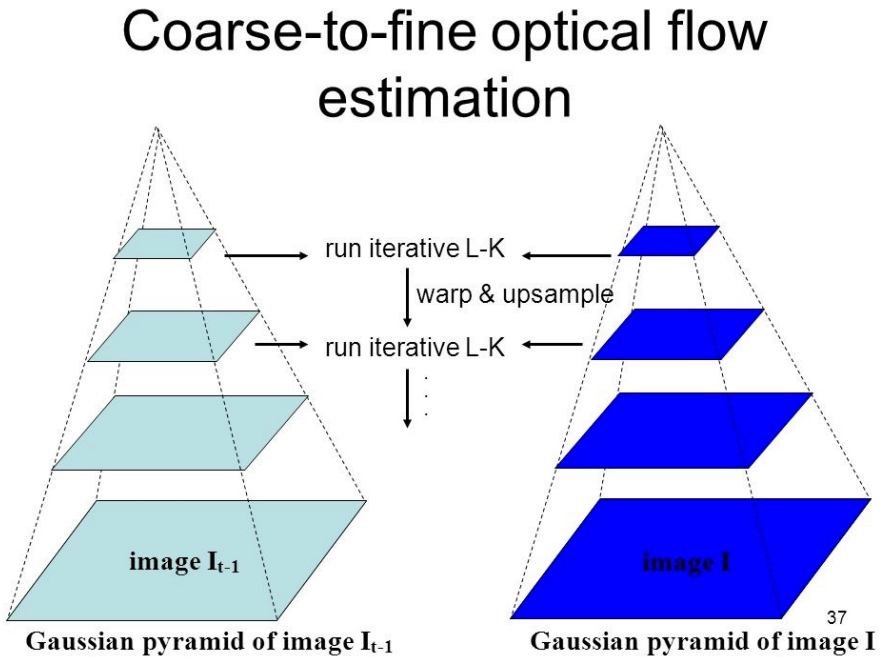
  - Then we get an overdetermined equation and solve it by linear least square (or Moore-Penrose inverse of the coefficient matrix:

$$\boldsymbol{A} = \begin{bmatrix} [\boldsymbol{I}_x, \boldsymbol{I}_y]_1 \\ \vdots \\ [\boldsymbol{I}_x, \boldsymbol{I}_y]_k \end{bmatrix}, \boldsymbol{b} = \begin{bmatrix} \boldsymbol{I}_{t1} \\ \vdots \\ \boldsymbol{I}_{tk} \end{bmatrix}. \qquad \begin{bmatrix} u \\ v \end{bmatrix}^* = -\left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{b}.$$

2. Optical Flow

- The solution of LK flow depends on the image gradient
  - Which is not smooth and can have dramatic changes
  - Multi-level optical flow: from coarse to fine

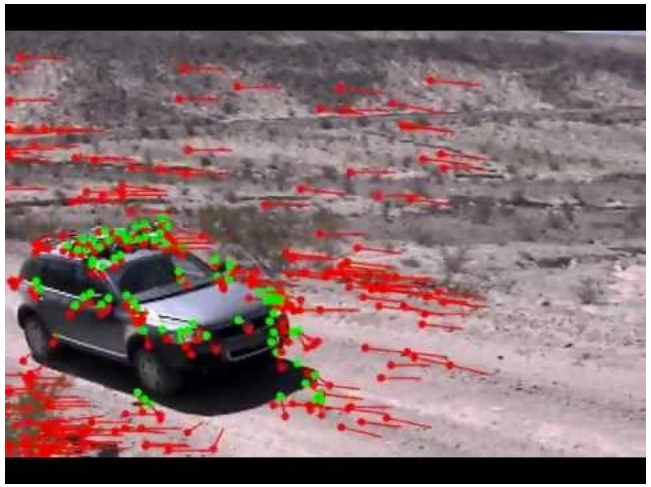
$$\frac{\partial \boldsymbol{I}}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t}+\frac{\partial \boldsymbol{I}}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t}=-\frac{\partial \boldsymbol{I}}{\partial t}.$$



2. Optical Flow

- Some notes on LK-flow
  - We can also use non-linear optimization tools (Gauss-Newton, L-M, etc.) to solve the optical flow iteratively
  - Optical flow can be used to track the motion of the corners in videos
  - After obtaining the points, motion estimation step will be same as feature methods
  - Need to wrap the patches if the motion is not pure translation





Contents

- Feature extraction and matching
- Optical Flow
- Pose estimation approaches
- Direct method

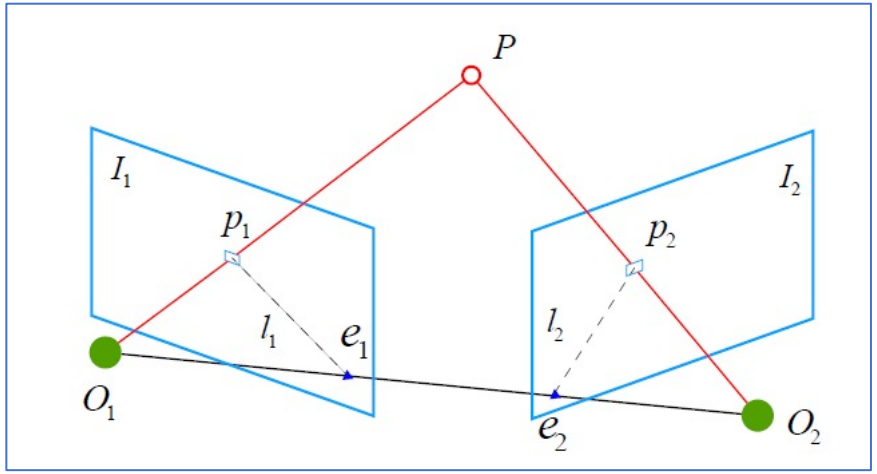
3. Pose estimation approaches

- After obtaining the corresponding feature points we can estimate the camera motion
- In practice we have several cases:
  - 2D-2D: if we only have two images
  - 3D-2D: if we have a pre-built scene and an image
  - 3D-3D: if we have two RGB-D image pairs or if we want to align a model to a scene

- 2D-2D: epipolar geometry
- 3D-2D: Perspective-n-Points (PnP)
- 3D-3D: iterative closest points (ICP)

3. Pose estimation approaches

- 2D-2D
  - 3D point P (unknown)
  - Projections: p1, p2
  - Images I1, I2
  - Transform: T12



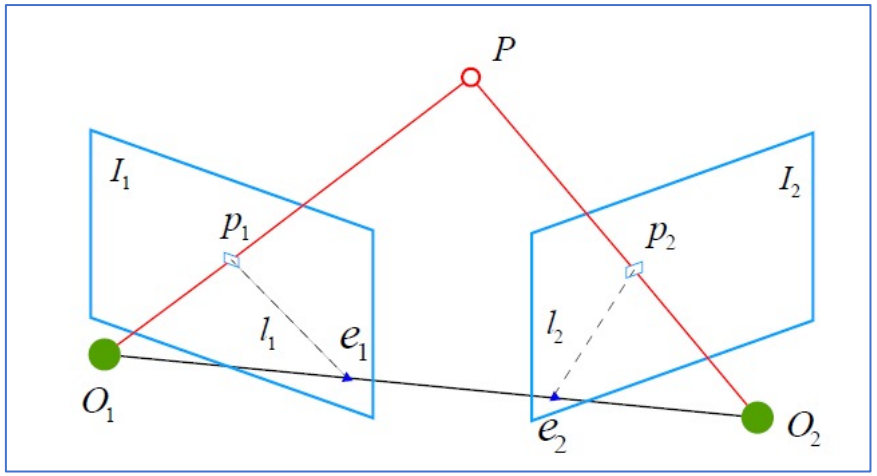
- Epipoles: projection of O_1, O_2 -> e_1, e_2
- Epipolar line: O_1P projected in image 2 e_2p_2 and vise vesa

- Our purpose: estimate T12(T21) given p1 and p2

3. Pose estimation approaches

- Epipolar geometry
- Assume the point P at $P = [X, Y, Z]^T.$
- Projection model:

$$s_1 p_1 = KP, \quad s_2 p_2 = K(RP + t).$$



- Use unit plane homogenous coordinates (to remove the intrinsics):

$$x_1 = K^{-1} p_1, \quad x_2 = K^{-1} p_2. \quad \Rightarrow \quad x_2 = R x_1 + t.$$

- Left multiplied by $t^\wedge$: $t^\wedge x_2 = t^\wedge R x_1.$
- Then left multiplied by x_2^T : $x_2^T t^\wedge x_2 = x_2^T t^\wedge R x_1.$

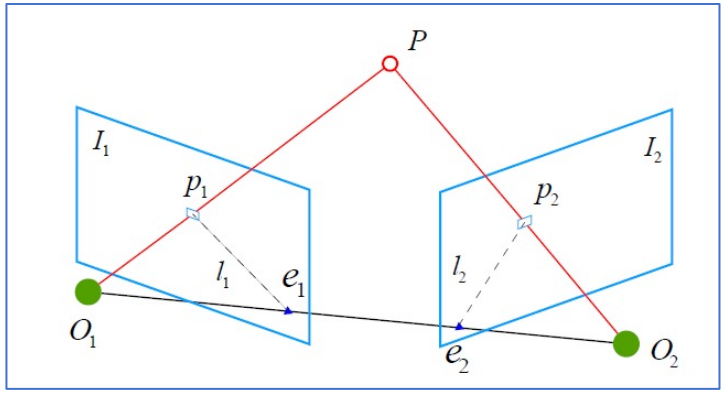
- This should be zero: $x_2^T t^\wedge R x_1 = 0.$

3. Pose estimation approaches

- Epipolar constraints: $x_2^T t^\wedge R x_1 = 0.$

$$p_2^T K^{-T} t^\wedge R K^{-1} p_1 = 0.$$

- Which meas O1, O2 and P are in the same plane
- Define: $E = t^\wedge R, \quad F = K^{-T} E K^{-1}, \quad x_2^T E x_1 = p_2^T F p_1 = 0.$
  - Essential/Fundamental matrix



- Estimate the pose in two steps:
  - Estimate Essential/Fundamental matrix
  - Decompose the E/F to get R,t

3. Pose estimation approaches

- Estimate essential matrix (eight-point algorithm)
  - Treat E as an ordinary matrix, then one point gives us:

$$\begin{pmatrix} u_1, v_1, 1 \end{pmatrix} \begin{pmatrix} e_1 & e_2 & e_3 \\ e_4 & e_5 & e_6 \\ e_7 & e_8 & e_9 \end{pmatrix} \begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} = 0.$$

  - Rewrite it as: $[u_1u_2, u_1v_2, u_1, v_1u_2, v_1v_2, u_2, v_2, 1] \cdot e = 0.$
  - Then we need at least eight points to solve this linear equation (because E is homogenous and can be multiplied by any non-zero factor)

$$\begin{pmatrix} u_1^1u_2^1 & u_1^1v_2^1 & u_1^1 & v_1^1u_2^1 & v_1^1v_2^1 & v_1^1 & u_2^1 & v_2^1 & 1 \\ u_1^2u_2^2 & u_1^2v_2^2 & u_1^2 & v_1^2u_2^2 & v_1^2v_2^2 & v_1^2 & u_2^2 & v_2^2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \\ u_1^8u_2^8 & u_1^8v_2^8 & u_1^8 & v_1^8u_2^8 & v_1^8v_2^8 & v_1^8 & u_2^8 & v_2^8 & 1 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \\ e_6 \\ e_7 \\ e_8 \\ e_9 \end{pmatrix} = 0.$$

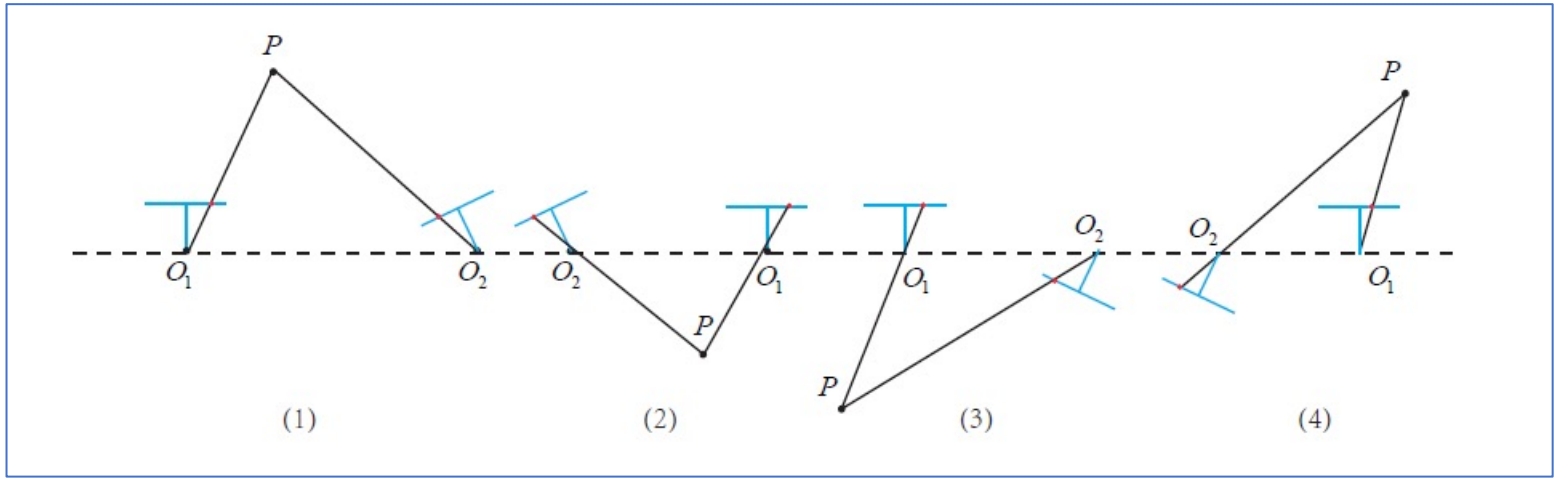
3. Pose estimation approaches

- From Essential to R,t: use the SVD method

$$E = U\Sigma V^T,$$

$$t_1^\wedge = UR_Z(\tfrac{\pi}{2})\Sigma U^T, \quad R_1 = UR_Z^T(\tfrac{\pi}{2})V^T$$
$$t_2^\wedge = UR_Z(-\tfrac{\pi}{2})\Sigma U^T, \quad R_2 = UR_Z^T(-\tfrac{\pi}{2})V^T.$$

- Four possible solutions but only one of them has positive depth values



3. Pose estimation approaches

- During the SVD we can also:
  - Take $\boldsymbol{E} = \boldsymbol{U}\mathrm{diag}(\frac{\sigma_1+\sigma_2}{2}, \frac{\sigma_1+\sigma_2}{2}, 0)\boldsymbol{V}^T.$ since essential matrix requires its singular value as $\sigma, \sigma, 0$
- And because DoF of E is only five (3 rot+3 trans -1 scale), we can also solve it using only 5 points
  - Called five point algorithm [1]

- More than eight points:
  - RANSAC or least square

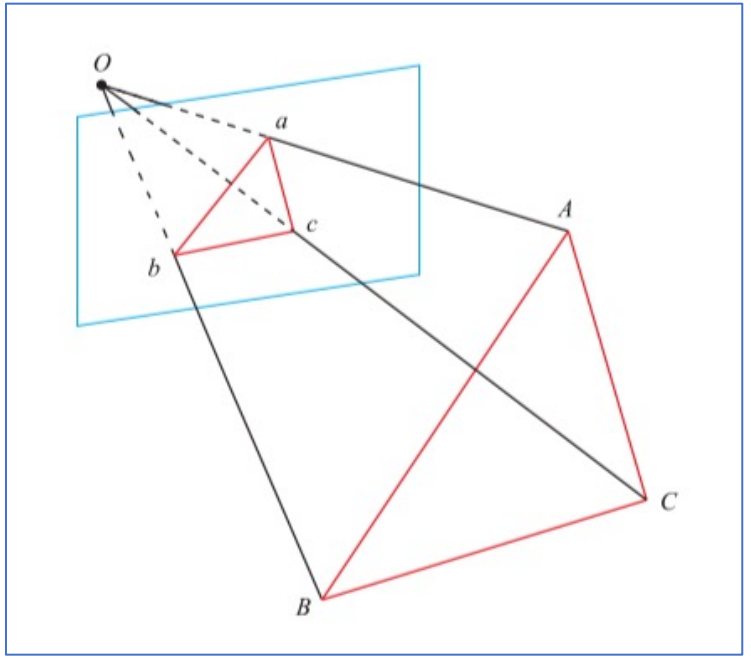
[1] Nistér D. An efficient solution to the five-point relative pose problem[J]. IEEE transactions on pattern analysis and machine intelligence, 2004, 26(6): 756-770.

3. Pose estimation approaches

- Eight-point algorithm can be used in the initialization step of monocular SLAM
- Some notes on Essential matrix
  - Scale is undetermined: we can normalize t or the mean depth of the scene
  - Pure rotation problem: when t=0 and t^R should be zero, and E cannot be decomposed
  - When the eight points are on the same plane, the problem will be degenerated and we will use Homography matrix to solve the initialization problem

3. Pose estimation approaches

- 3D-2D: Perspective-n-Points
- Given n 3D points and their projections, estimate the camera pose
- Methods: linear algebra or nonlinear optimization
- Linear algebra:
  - DLT
  - P3P
  - EPnP/UPnP/etc.
  - You can just call cv::SolvePnP
  - The methods can be chosen by giving different parameters
- Nonlinear:
  - Minimizing the reprojection error
  - Shown in ex3.3



3. Pose estimation approaches

- 3D-3D: ICP
- Given two pairs of 3D points, estimate the rotation and translation
- The points can be pre-matched or not matched
- If we have the matches, then the problem has analytical solution, otherwise we don’ t
- Assume we have two point sets: $\boldsymbol{P} = \{\boldsymbol{p}_1, \cdots, \boldsymbol{p}_n\}, \quad \boldsymbol{P}' = \{\boldsymbol{p}'_1, \cdots, \boldsymbol{p}'_n\},$
- And the motion is: $\forall i, \boldsymbol{p}_i = \boldsymbol{R}\boldsymbol{p}'_i + \boldsymbol{t}.$
- Define the error: $\boldsymbol{e}_i = \boldsymbol{p}_i - (\boldsymbol{R}\boldsymbol{p}'_i + \boldsymbol{t}).$
- And the least square will be: $$\min_{\boldsymbol{R},\boldsymbol{t}} J = \frac{1}{2}\sum_{i=1}^{n} \|(\boldsymbol{p}_i - (\boldsymbol{R}\boldsymbol{p}_i{}' + \boldsymbol{t}))\|_2^2.$$

3. Pose estimation approaches

- Some derivation
- Define the centroids: $p = \frac{1}{n}\sum_{i=1}^{n}(p_i), \quad p' = \frac{1}{n}\sum_{i=1}^{n}(p_i').$
- And rewrite the objective function:

$$
\begin{aligned}
\frac{1}{2}\sum_{i=1}^{n}\left\|\boldsymbol{p}_i-(\boldsymbol{R}\boldsymbol{p}_i{}'+\boldsymbol{t})\right\|^2 &= \frac{1}{2}\sum_{i=1}^{n}\left\|\boldsymbol{p}_i-\boldsymbol{R}\boldsymbol{p}_i{}'-\boldsymbol{t}-\boldsymbol{p}+\boldsymbol{R}\boldsymbol{p}'+\boldsymbol{p}-\boldsymbol{R}\boldsymbol{p}'\right\|^2 \\
&= \frac{1}{2}\sum_{i=1}^{n}\left\|\left(\boldsymbol{p}_i-\boldsymbol{p}-\boldsymbol{R}\left(\boldsymbol{p}_i{}'-\boldsymbol{p}'\right)\right)+\left(\boldsymbol{p}-\boldsymbol{R}\boldsymbol{p}'-\boldsymbol{t}\right)\right\|^2 \\
&= \frac{1}{2}\sum_{i=1}^{n}\left(\left\|\boldsymbol{p}_i-\boldsymbol{p}-\boldsymbol{R}\left(\boldsymbol{p}_i{}'-\boldsymbol{p}'\right)\right\|^2+\left\|\boldsymbol{p}-\boldsymbol{R}\boldsymbol{p}'-\boldsymbol{t}\right\|^2+\right. \\
&\qquad \left.2\left(\boldsymbol{p}_i-\boldsymbol{p}-\boldsymbol{R}\left(\boldsymbol{p}_i{}'-\boldsymbol{p}'\right)\right)^T\left(\boldsymbol{p}-\boldsymbol{R}\boldsymbol{p}'-\boldsymbol{t}\right)\right).
\end{aligned}
$$

This part is zero if you write it out

3. Pose estimation approaches

- So the objective function is simplified as:

$$\min_{\boldsymbol{R},\boldsymbol{t}} J = \frac{1}{2}\sum_{i=1}^{n} \left\| \boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}\left(\boldsymbol{p}_i{}' - \boldsymbol{p}'\right)\right\|^2 + \left\|\boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' - \boldsymbol{t}\right\|^2.$$

- We can just minimize the first part, and choose a t to set the second part to zero
- How to solve the first part?
- Remove the centroid: $\boldsymbol{q}_i = \boldsymbol{p}_i - \boldsymbol{p}, \quad \boldsymbol{q}_i' = \boldsymbol{p}_i' - \boldsymbol{p}'.$
- The problem becomes: $\boldsymbol{R}^* = \arg\min_{\boldsymbol{R}} \frac{1}{2}\sum_{i=1}^{n} \left\|\boldsymbol{q}_i - \boldsymbol{R}\boldsymbol{q}_i'\right\|^2.$

3. Pose estimation approaches

- Some derivation:

$$\frac{1}{2}\sum_{i=1}^{n}\left\|\boldsymbol{q}_i-\boldsymbol{R}\boldsymbol{q}_i'\right\|^2=\frac{1}{2}\sum_{i=1}^{n}\boldsymbol{q}_i^{\mathrm{T}}\boldsymbol{q}_i+\boldsymbol{q}_i'^{\mathrm{T}}\boldsymbol{R}^{\mathrm{T}}\boldsymbol{R}\boldsymbol{q}_i'-2\boldsymbol{q}_i^{\mathrm{T}}\boldsymbol{R}\boldsymbol{q}_i'.$$

- It’ s only related to the last part:

$$\sum_{i=1}^{n}-\boldsymbol{q}_i^{\mathrm{T}}\boldsymbol{R}\boldsymbol{q}_i'=\sum_{i=1}^{n}-\mathrm{tr}\left(\boldsymbol{R}\boldsymbol{q}_i'\boldsymbol{q}_i^{\mathrm{T}}\right)=-\mathrm{tr}\left(\boldsymbol{R}\sum_{i=1}^{n}\boldsymbol{q}_i'\boldsymbol{q}_i^{\mathrm{T}}\right).$$

- Solve it by SVD:

$$\boldsymbol{W}=\sum_{i=1}^{n}\boldsymbol{q}_i\boldsymbol{q}_i'^{\mathrm{T}}. \qquad \boldsymbol{W}=\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}. \qquad \boldsymbol{R}=\boldsymbol{U}\boldsymbol{V}^{\mathrm{T}}.$$

3. Pose estimation approaches

- It can be proven that if we find a solution, then this solution is the global minimal
- Otherwise, in some special degenerated cases, we cannot find a solution

- If we don’ t have matches, then assume the closes points are matches and solve this ICP iteratively
- Also, note in the RGB-D case, you can use ICP and PnP separately or put them together into a Bundle Adjustment

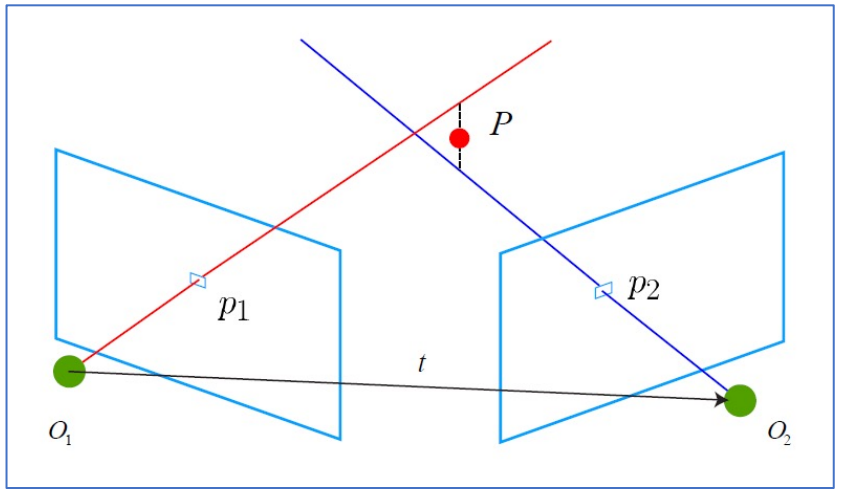
3. Pose estimation approaches

- Triangulation
- Given the motion and the pixels, estimate the 3D point position
- From geometry we know: $s_1 x_1 = s_2 R x_2 + t.$
- We can either solve s1 or s2, take s2 as an example
- Left multiply by $x_1^\wedge$:

$$s_1 x_1^\wedge x_1 = 0 = s_2 x_1^\wedge R x_2 + x_1^\wedge t.$$

  - Overdetermined linear equation
- We can also solve s1, s2 together:

$$[-Rx_2, x_1] \begin{bmatrix} s_1 \\ s_2 \end{bmatrix} = t$$



3. Pose estimation approaches

- Summary
  - We can get point pairs by feature matching or optical flow
  - Estimate the pose in some cases:
    - 2D-2D: estimate the essential/fundamental/Homography matrix and then solve R,t from them
    - 3D-2D: PnP, linear and non-linear method
    - 3D-3D: ICP, also linear (SVD) method or nonlinear method

Contents

- Feature extraction and matching
- Optical Flow
- Pose estimation approaches
- Direct method

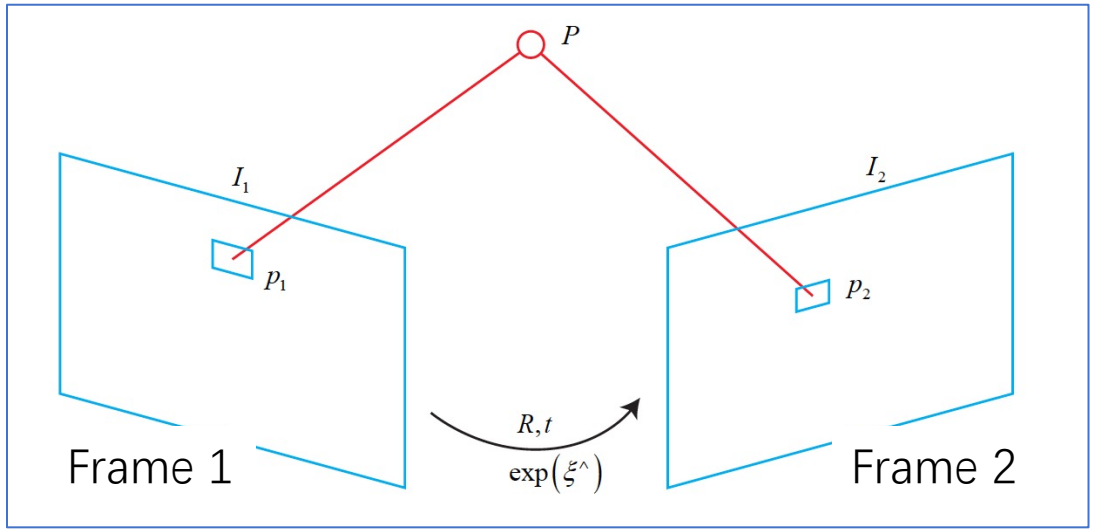
4. Direct Method

- Optical flow can estimate pixel’ s motion, but
  - Without considering the camera’ s projection model
- Direct method: put them together in an optimization problem

4. Direct Method

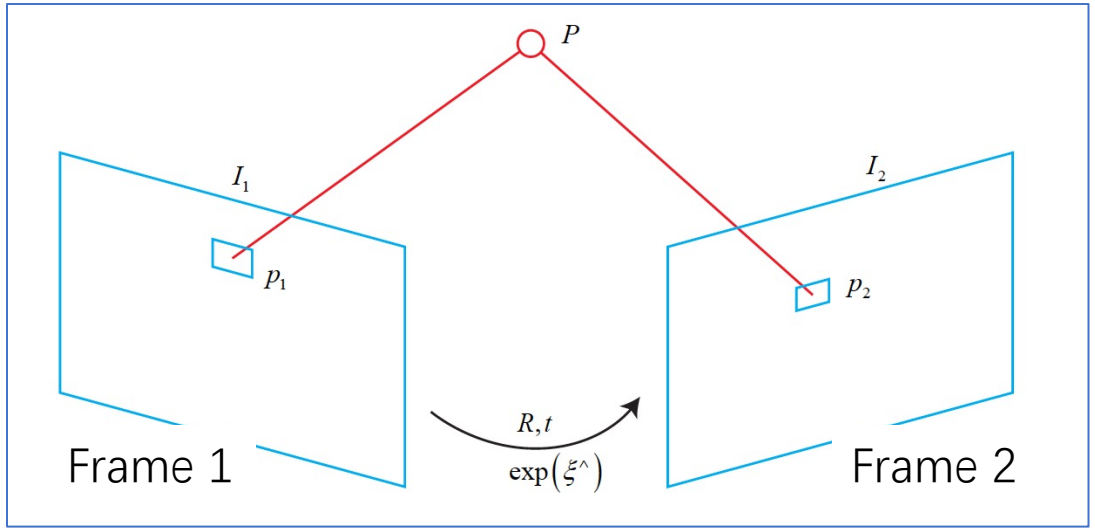
- Derivation of direct method
  - Assume we have two image and the motion is unknown (but with a initial guess)
  - We have a pixel at image 1, and know its depth
  - Then we can compute the projection using the initial guess of transform

$$s_1 p_1 = KP$$
$$s_2 p_2 = K(RP + t)$$



4. Direct Method

- Brightness constancy assumption: $I_1(p_1) = I_2(p_2)$
- Brightness error: $e = I_1(p_1) - I_2(p_2)$
- Motion estimation by least square: $\min_T J(T) = \sum_{i=1}^{N} e_i^T e_i$



4. Direct Method

- Jacobians by disturb model in SE(3)

$$e(\delta T \oplus T) = I_1(p_1) - I_2\left(\frac{1}{Z_2} K \exp(\delta\xi\hat{})TP\right)$$

$$\approx I_1(p_1) - I_2\left(\frac{1}{Z_2} K(1 + \delta\xi\hat{})TP\right)$$

$$= I_1(p_1) - I_2\left(p_2 + \frac{1}{Z_2} K\delta\xi\hat{}TP\right)$$

Define:

$$q = \delta\xi\hat{}TP, \qquad u = \frac{1}{Z_2} Kq$$

Then:

$$e(\delta T \oplus T) = I_1(p_1) - I_2(p_2 + u)$$

$$\approx I_1(p_1) - I_2(p_2) - \frac{\partial I_2}{\partial u}\frac{\partial u}{\partial q}\frac{\partial q}{\partial \delta\xi}\delta\xi$$

$$= e - \frac{\partial I_2}{\partial u}\frac{\partial u}{\partial q}\frac{\partial q}{\partial \delta\xi}\delta\xi$$

4. Direct Method

- So Jacobian has three parts:
  - First $\frac{\partial I_2}{\partial u}$ is the image gradients in the second image
  - And the second the third part is same as geometric reprojection error:

$$\frac{\partial \boldsymbol{u}}{\partial \delta \boldsymbol{\xi}} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & -\frac{f_x XY}{Z^2} & f_x + \frac{f_x X^2}{Z^2} & -\frac{f_x Y}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & -f_y - \frac{f_y Y^2}{Z^2} & \frac{f_y XY}{Z^2} & \frac{f_y X}{Z} \end{bmatrix}.$$

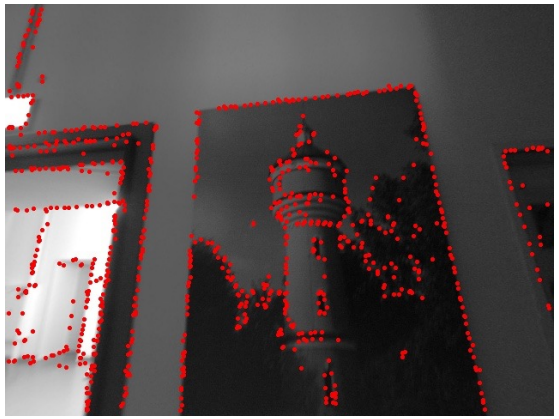
  - So the overall Jacobian will be:

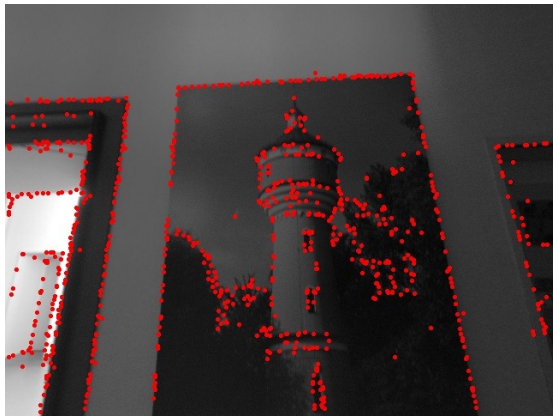
$$\boldsymbol{J} = -\frac{\partial \boldsymbol{I}_2}{\partial \boldsymbol{u}} \frac{\partial \boldsymbol{u}}{\partial \delta \boldsymbol{\xi}}.$$

- So we need to choose points that have non-zero gradients, otherwise they won’ t contribute to the pose estimation

4. Direct Method

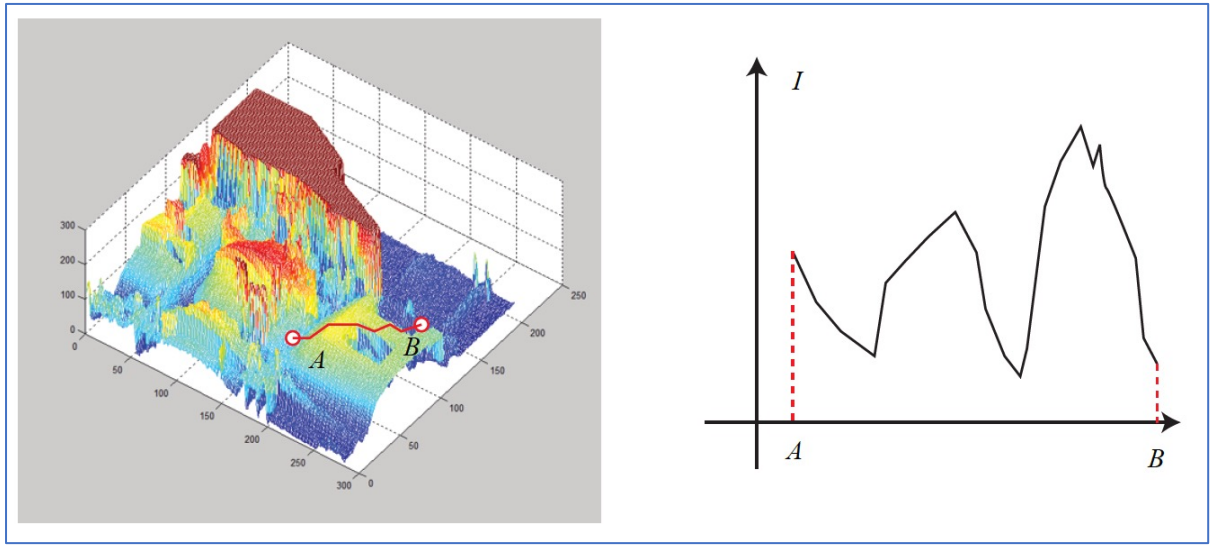
- Some notes on direct method
  - We need to know the depth in the first image (or the reference image), which can be obtained from an RGB-D camera or pre-built structures
  - Don’ t need explicit matched points, what we only need is image gradients (thus we can choose edges and smooth areas)
  - Are able to build dense or semi-dense maps
  - Can be also extended to multi-level and lead to a coarse-to-fine direct method





4. Direct Method

- Direct method is also affected by image gradients
  - We usually can not control or predict the image data
  - So if the motion is too large, we can not guarantee the cost function is always decreasing during the path to the correct point
  - So direct method is only suitable for smooth motion (or high speed camera)



What we will cover in this course?

- Lecture 1. Preliminary Knowledge
- Lecture 2. Optimization on Manifold
- Lecture 3. Visual Odometry
- Lecture 4. Backend Optimization

Contents

- Recursive Optimization
- Batch Optimization
- Pose graph

Contents

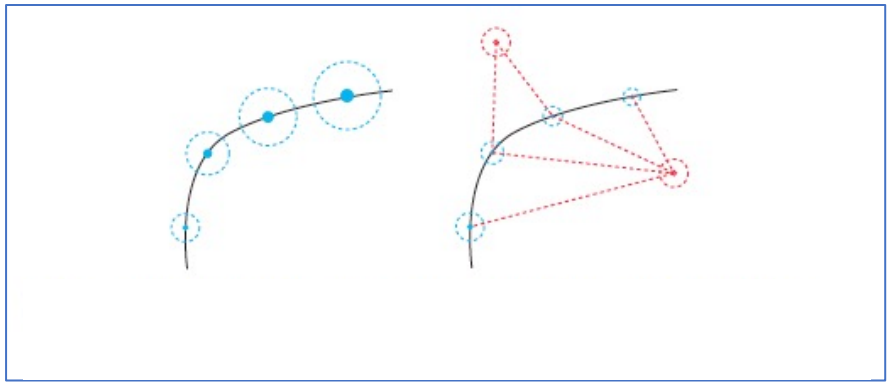
- Recursive Optimization
- Batch Optimization
- Pose graph

1. Recursive Optimization

- Backend
  - Estimate the state variables from the noisy data
- Batch way
  - Estimate the best state given all the data
  - Bundle Adjustment in visual SLAM system
- Incremental way
  - Keep the current (best) estimation, update it when new data is arrived
  - Also throw away the past information
  - Kalman Filter: Linear system + Gaussian noise
  - Extended Kalman Filter: Nonlinear system + Gaussian noise
  - Sliding window filter & multiple state constraint Kalman Filter

1. Recursive Optimization

- A simple example
- When we walk blindfolded:
  - At the beginning we know where we are
  - Roughly estimate the distance of each step
  - Uncertainty accumulates over time
- When you open your eyes at some time:
  - Can observe the soundings
  - Uncertainty in each step is still the same
  - But can be corrected by observation
  - Overall uncertainty can be kept within a certain range



1. Recursive Optimization

- Recall the motion and observation model:

$$\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k\right) + \boldsymbol{w}_k \\ \boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k\right) + \boldsymbol{v}_{k,j} \end{cases} \quad k = 1, \ldots, N,\ j = 1, \ldots, M.$$

- Let’ s start from Bayes filter
- Use x_k to denote the unknown variables in time k:

$$\boldsymbol{x}_k \triangleq \{\boldsymbol{x}_k, \boldsymbol{y}_1, \ldots, \boldsymbol{y}_m\}.$$

- Then the model can be simplified as $\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k\right) + \boldsymbol{w}_k \\ \boldsymbol{z}_k = h\left(\boldsymbol{x}_k\right) + \boldsymbol{v}_k \end{cases} \quad k = 1, \ldots, N.$

1. Recursive Optimization

- We show how to derive the recursive approach from batch approach
- Estimate the current state given data from 0 to k: $P(\boldsymbol{x}_k|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k})$.

- Bayes' rule (switch z_k): Likehood Prior

$$P(\boldsymbol{x}_k|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k}) \propto P(\boldsymbol{z}_k|\boldsymbol{x}_k) P(\boldsymbol{x}_k|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}).$$

- Expand the prior:

$$P(\boldsymbol{x}_k|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) = \int P(\boldsymbol{x}_k|\boldsymbol{x}_{k-1}, \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) P(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) \mathrm{d}\boldsymbol{x}_{k-1}. \quad (10.6)$$

Motion model prediction Estimation in k-1

1. Recursive Optimization

$$P(\boldsymbol{x}_k|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) = \int P(\boldsymbol{x}_k|\boldsymbol{x}_{k-1}, \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) P(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) \, \mathrm{d}\boldsymbol{x}_{k-1}. \tag{10.6}$$

- Different ways to treat this equation:
  - Assume the Markov property: we assume x_k is only relevant to x_k-1
  - Don’ t assume Markov property: x_k is relevant to all previous state

1. Recursive Optimization

- By assuming the Markov’ s property:

$$P\left(\boldsymbol{x}_k|\boldsymbol{x}_{k-1},\boldsymbol{x}_0,\boldsymbol{u}_{1:k},\boldsymbol{z}_{1:k-1}\right)=P\left(\boldsymbol{x}_k|\boldsymbol{x}_{k-1},\boldsymbol{u}_k\right).$$

- The second item becomes:

$$P\left(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0,\boldsymbol{u}_{1:k},\boldsymbol{z}_{1:k-1}\right)=P\left(\boldsymbol{x}_{k-1}|\boldsymbol{x}_0,\boldsymbol{u}_{1:k-1},\boldsymbol{z}_{1:k-1}\right).$$

- This equation (Bayes’ rule) shows how to recursively estimate the status
  - But we haven’ t set the specific form of motion and obs model
- In Linear-Gaussian (LG) system, the recursive approach will lead to Kalman Filter (KF)

1. Recursive Optimization

- Derivation of KF in LG system

$$\begin{cases} \boldsymbol{x}_k = \boldsymbol{A}_k \boldsymbol{x}_{k-1} + \boldsymbol{u}_k + \boldsymbol{w}_k \\ \boldsymbol{z}_k = \boldsymbol{C}_k \boldsymbol{x}_k + \boldsymbol{v}_k \end{cases} \quad k = 1, \ldots, N.$$

$\boldsymbol{w}_k \sim N(\boldsymbol{0}, \boldsymbol{R}). \quad \boldsymbol{v}_k \sim N(\boldsymbol{0}, \boldsymbol{Q}).$ are noises

- Assume the state variables are Gaussian

$$P(x_{k-1}) = N\left(\hat{x}_{k-1}, \hat{P}_{k-1}\right) \qquad \overline{x}_k, \overline{P}_k$$

Posterior Prior

- Use different notations since we need Bayes' rule

1. Recursive Optimization

- Some conclusions to start with:
- Linear transform of Gaussian distribution:
  - Assume $x \sim N(\mu, \Sigma), y = Ax + b$, then y is also Gaussian and satisfies:

$$
\begin{aligned}
E[y] &= E[Ax + b] = AE[x] + b = A\mu + b \\
Cov[y] &= E[(y - E[y])(y - E[y])^T] \\
&= E[A(x - \mu)(x - \mu)^T A^T] = A\Sigma A^T
\end{aligned}
$$

1. Recursive Optimization

- With this we can derive the prior at time k using motion model:

$$\begin{cases} \boldsymbol{x}_k = \boldsymbol{A}_k \boldsymbol{x}_{k-1} + \boldsymbol{u}_k + \boldsymbol{w}_k \\ \boldsymbol{z}_k = \boldsymbol{C}_k \boldsymbol{x}_k + \boldsymbol{v}_k \end{cases} \quad k = 1, \ldots, N.$$

- With motion model: $P(\boldsymbol{x}_k | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}) = N\left(\boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{u}_k, \boldsymbol{A}_k \hat{\boldsymbol{P}}_k \boldsymbol{A}_k^{\mathrm{T}} + \boldsymbol{R}\right).$

- This equation gives the prior distribution, denoted it as:

$$\bar{\boldsymbol{x}}_k = \boldsymbol{A}_k \hat{\boldsymbol{x}}_{k-1} + \boldsymbol{u}_k, \quad \bar{\boldsymbol{P}}_k = \boldsymbol{A}_k \hat{\boldsymbol{P}}_{k-1} \boldsymbol{A}_k^{\mathrm{T}} + \boldsymbol{R}.$$

- From observation model we know:

$$P(z_k \mid x_k) = N(C_k x_k, Q).$$

- Compute the posterior model:

$$P(\boldsymbol{x}_k | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k}) \propto P(\boldsymbol{z}_k | \boldsymbol{x}_k) P(\boldsymbol{x}_k | \boldsymbol{x}_0, \boldsymbol{u}_{1:k}, \boldsymbol{z}_{1:k-1}).$$

1. Recursive Optimization

- A small trick: we assume the posterior is also Gaussian, so:

$$N(\hat{x}_k, \hat{P}_k) = \eta N(C_k x_k, Q_k) \cdot N(\bar{x}_k, \bar{P}_k)$$

- Since they are all Gaussian, so we just expand it and compare the linear and quadratic coefficients
- The exponential part is:

$$\left(x_k - \hat{x}_k\right)^{\mathrm{T}} \hat{P}_k^{-1} \left(x_k - \hat{x}_k\right) = \left(z_k - C_k x_k\right)^{\mathrm{T}} Q^{-1} \left(z_k - C_k x_k\right) + \left(x_k - \bar{x}_k\right)^{\mathrm{T}} \bar{P}_k^{-1} \left(x_k - \bar{x}_k\right).$$

- Compare the coefficients of x_k , for the quadratic part we have:

$$\boxed{\hat{P}_k^{-1} = C_k^{\mathrm{T}} Q^{-1} C_k + \bar{P}_k^{-1}.}$$

1. Recursive Optimization

- For the linear part we have:

$$\left(x_k-\hat{x}_k\right)^{\mathrm{T}}\hat{P}_k^{-1}\left(x_k-\hat{x}_k\right)=\left(z_k-C_kx_k\right)^{\mathrm{T}}Q^{-1}\left(z_k-C_kx_k\right)+\left(x_k-\bar{x}_k\right)^{\mathrm{T}}\bar{P}_k^{-1}\left(x_k-\bar{x}_k\right).$$

$$-2\hat{x}_k^{\mathrm{T}}\hat{P}_k^{-1}x_k=-2z_k^{\mathrm{T}}Q^{-1}C_kx_k-2\bar{x}_k^{\mathrm{T}}\ \bar{P}_k^{-1}x_k$$

- Rearrange it: $\boxed{\hat{P}_k^{-1}\hat{x}_k=C_k^{\mathrm{T}}Q^{-1}z_k+\bar{P}_k^{-1}\bar{x}_k}$
- Left multiply $\hat{P}_k$ and define: $K=\hat{P}_k\ C_k^{\mathrm{T}}\ Q^{-1}$, then we have:

$$\begin{aligned}\hat{x}_k &= \hat{P}_k\ C_k^{\mathrm{T}}\ Q^{-1}z_k+\hat{P}_k\bar{P}_k^{-1}\bar{x}_k \\ &= Kz_k+\left(I-KC_k\right)\bar{x}_k=\bar{x}_k+K\left(z_k-C_k\bar{x}_k\right).\end{aligned}$$

Innovation part

$$\boxed{\hat{P}_k^{-1}=\ C_k^{\mathrm{T}}Q^{-1}C_k+\bar{P}_k^{-1}.}$$

K: Kalman gain

1. Recursive Optimization

- Kalman gain: $K = \hat{P}_k \, C_k^{\mathrm{T}} \, Q^{-1}$ requires $\hat{P}_k$
- Another form:

$$K = \left(C_k^T Q^{-1} C_k + \bar{P}_k^{-1}\right)^{-1} C_k^T Q^{-1}$$
$$= \bar{P}_k C_k^T (Q + C_k \bar{P}_k C^T)^{-1}$$

- This

$$\left(\boldsymbol{A}^{-1} + \boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}\right)^{-1} \equiv \boldsymbol{A} - \boldsymbol{A}\boldsymbol{B}\left(\boldsymbol{D} + \boldsymbol{C}\boldsymbol{A}\boldsymbol{B}\right)^{-1}\boldsymbol{C}\boldsymbol{A} \quad (2.75a)$$

$$\left(\boldsymbol{D} + \boldsymbol{C}\boldsymbol{A}\boldsymbol{B}\right)^{-1} \equiv \boldsymbol{D}^{-1} - \boldsymbol{D}^{-1}\boldsymbol{C}\left(\boldsymbol{A}^{-1} + \boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}\right)^{-1}\boldsymbol{B}\boldsymbol{D}^{-1} \quad (2.75b)$$

$$\boldsymbol{A}\boldsymbol{B}\left(\boldsymbol{D} + \boldsymbol{C}\boldsymbol{A}\boldsymbol{B}\right)^{-1} \equiv \left(\boldsymbol{A}^{-1} + \boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}\right)^{-1}\boldsymbol{B}\boldsymbol{D}^{-1} \quad (2.75c)$$

$$\left(\boldsymbol{D} + \boldsymbol{C}\boldsymbol{A}\boldsymbol{B}\right)^{-1}\boldsymbol{C}\boldsymbol{A} \equiv \boldsymbol{D}^{-1}\boldsymbol{C}\left(\boldsymbol{A}^{-1} + \boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}\right)^{-1} \quad (2.75d)$$

1. Recursive Optimization

- Two steps in Kalman filter

1. Prediction $\bar{x}_k = A_k \hat{x}_{k-1} + u_k, \qquad \bar{P}_k = A_k \hat{P}_{k-1} A_k^T + R$
2. Correction
   - Compute Kalman gain: $K = \bar{P}_k C_k^T (Q + C_k \bar{P}_k C^T)^{-1}$
   - Update the estimation: $\hat{x}_k = \bar{x}_k + K(z_k - C_k \bar{x}_k)$
     $\hat{P}_k = (I - K C_k) \bar{P}_k$

1. Recursive Optimization

- Some notes on Kalman filter
  - Kalman filter is the BLUE (best linear unbiased estimate) estimation in LG system
  - Kalman filter gives the same result as MAP in LG system
    - This is because the mode and mean are same in Gauss distribution
    - We can also derive KF through optimization way
    - Or by choose a best Kalman gain to get the best estimation

1. Recursive Optimization

- Extended KF in NL systems:

$$\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k\right) + \boldsymbol{w}_k \\ \boldsymbol{z}_k = h\left(\boldsymbol{x}_k\right) + \boldsymbol{v}_k \end{cases} \quad k = 1, \ldots, N.$$

- We take the Taylor expansion in current estimate:

$$\boldsymbol{x}_k \approx f\left(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k\right) + \left.\frac{\partial f}{\partial \boldsymbol{x}_{k-1}}\right|_{\hat{\boldsymbol{x}}_{k-1}} \left(\boldsymbol{x}_{k-1} - \hat{\boldsymbol{x}}_{k-1}\right) + \boldsymbol{w}_k.$$

Denoted as F

$$\boldsymbol{z}_k \approx h\left(\bar{\boldsymbol{x}}_k\right) + \left.\frac{\partial h}{\partial \boldsymbol{x}_k}\right|_{\bar{\boldsymbol{x}}_k} \left(\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k\right) + \boldsymbol{n}_k.$$

Denoted as H

1. Recursive Optimization

$$\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k\right) + \boldsymbol{w}_k \\ \boldsymbol{z}_k = h\left(\boldsymbol{x}_k\right) + \boldsymbol{v}_k \end{cases} \quad k = 1, \ldots, N.$$

- Then employ the conclusions in KF:
- Prediction: $\bar{\boldsymbol{x}}_k = f\left(\hat{\boldsymbol{x}}_{k-1}, \boldsymbol{u}_k\right), \quad \bar{\boldsymbol{P}}_k = \boldsymbol{F}\hat{\boldsymbol{P}}_k\boldsymbol{F}^{\mathrm{T}} + \boldsymbol{R}_k.$

- Correction:
  - Kalman gain: $\boldsymbol{K}_k = \bar{\boldsymbol{P}}_k\boldsymbol{H}^{\mathrm{T}}\left(\boldsymbol{H}\bar{\boldsymbol{P}}_k\boldsymbol{H}^{\mathrm{T}} + \boldsymbol{Q}_k\right)^{-1}.$
  - Update: $\hat{\boldsymbol{x}}_k = \bar{\boldsymbol{x}}_k + \boldsymbol{K}_k\left(\boldsymbol{z}_k - h\left(\bar{\boldsymbol{x}}_k\right)\right), \hat{\boldsymbol{P}}_k = \left(\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{H}\right)\bar{\boldsymbol{P}}_k.$

1. Recursive Optimization

- Discussion of KF and EKF
- Advantages
  - Clean and simple
  - Do not require any property of motion and observation model
  - Can be used for multiple sensor fusion
- Disadvantages
  - Need to assume Markov property (which is not satisfied when we have loop closure)
  - May diverge if the observations have outliers
  - Linearization may have error if the model has strong nonlinearity
  - Gaussian approximation may not be accurate for some variables
  - Need to store the mean and covariance matrix for all status

Contents

- Recursive Optimization
- Batch Optimization
- Pose graph

2. Batch Optimization

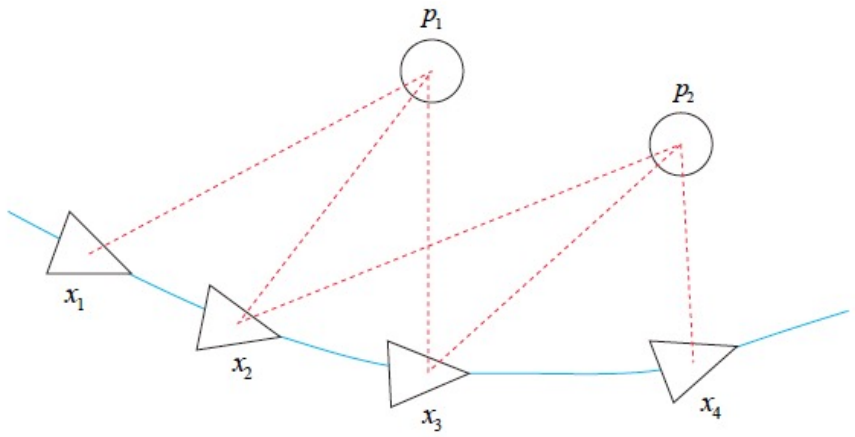
- Batch optimization
- We’ ve shown some conclusions in Lecture 3
- MAP estimation is equivalent to least square solution

$$\frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{n}\|\boldsymbol{e}_{ij}\|^2 = \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{n}\|\boldsymbol{z}_{ij} - h(\boldsymbol{\xi}_i, \boldsymbol{p}_j)\|^2.$$

- It is called Bundle Adjustment when used in visual SLAM systems
  - We have a bundle of lights and adjust the cameras to fit the observation model

2. Batch Optimization

- BA and graph optimization
  - Least square in BA can be represented as a graph G={V,E}
  - Where V is the node set containing the optimization variables
  - And E is the edge set containing the observation errors



Special pattern in BA:

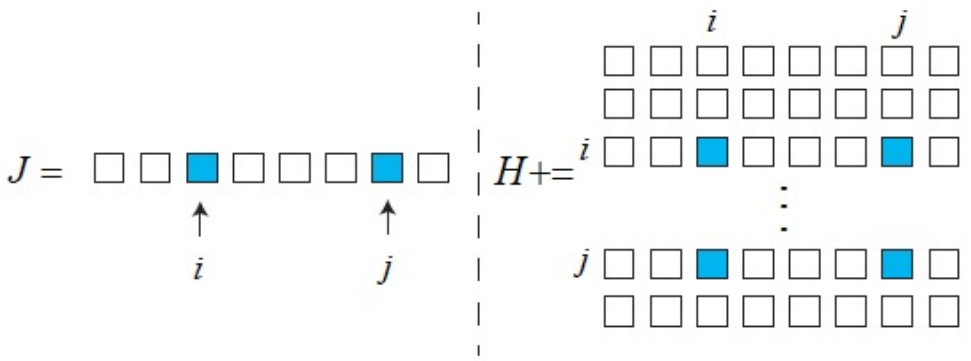
- Each observation is only related to two variables (nodes)
- We don’ t have point-point edges (structure prior)

2. Batch Optimization

- According to optimization theory we will finally need to solve the normal equation: $H\Delta x = -b$
- Each edge contributes to this H by: $H = \sum_{i,j} J_{ij}^T J_{ij}$
- Consider an observation regarding to i-th camera and j-th point:

$$\boldsymbol{J}_{ij}(\boldsymbol{x}) = \left(\mathbf{0}_{2\times 6}, \ldots \mathbf{0}_{2\times 6}, \frac{\partial \boldsymbol{e}_{ij}}{\partial \boldsymbol{\xi}_i}, \mathbf{0}_{2\times 6}, \ldots \mathbf{0}_{2\times 3}, \ldots \mathbf{0}_{2\times 3}, \frac{\partial \boldsymbol{e}_{ij}}{\partial \boldsymbol{p}_j}, \mathbf{0}_{2\times 3}, \ldots \mathbf{0}_{2\times 3}\right).$$

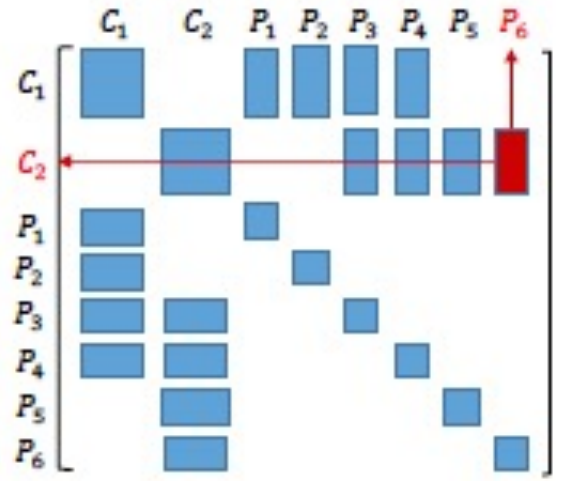
- This is a sparse matrix that only has two non-zero entries:

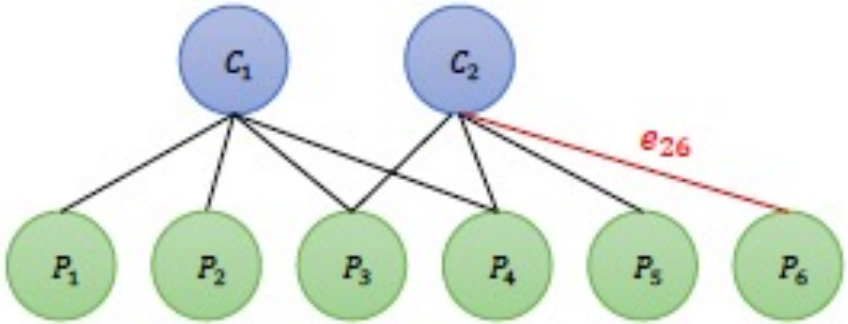


2. Batch Optimization

- If we set the order of the overall status by keeping the cameras at first and

$H =$

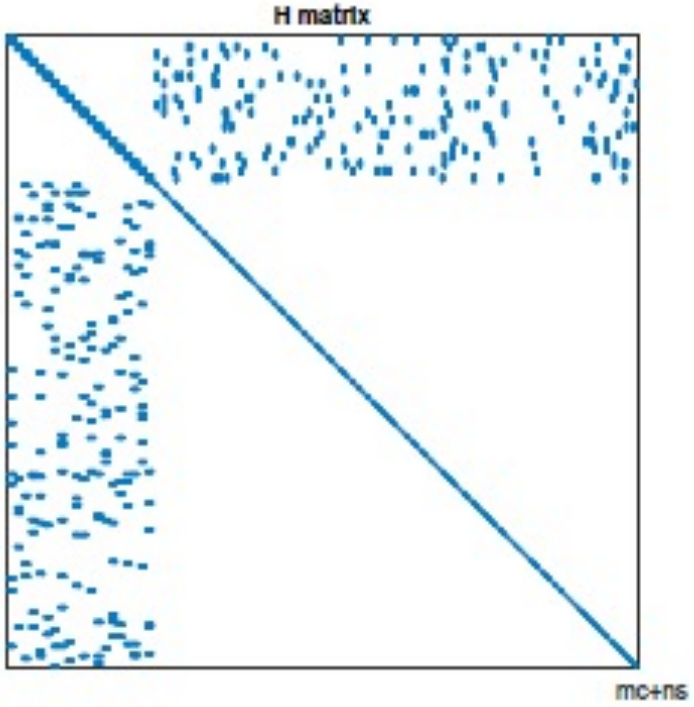




- The relationship of the graph and H matrix:
  - Each edge in the graph is corresponding to a non-zero block in H

2. Batch Optimization

- In real-world BA the number of points is far more than cameras, so the







The Arrow-like H matrix

2. Batch Optimization

- For a dense H matrix we need to inverse it to solve the normal equation, which has O(n^3) complexity
- But in BA this can be accelerated by employing the special structure of H
- Split the blocks in H:

$$\begin{bmatrix} \boldsymbol{B} & \boldsymbol{E} \\ \boldsymbol{E}^{\mathrm{T}} & \boldsymbol{C} \end{bmatrix} \begin{bmatrix} \Delta \boldsymbol{x}_c \\ \Delta \boldsymbol{x}_p \end{bmatrix} = \begin{bmatrix} \boldsymbol{v} \\ \boldsymbol{w} \end{bmatrix}.$$

B and C are diagonal block matrices
E and E^T is dense and the non-zero blocks are corresponding to real observations

- Idea:
  - Since C is block diagonal, we use Gaussian elimination to eliminate the E and E^T

$$\begin{bmatrix} \boldsymbol{I} & -\boldsymbol{E}\boldsymbol{C}^{-1} \\ \boldsymbol{0} & \boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{B} & \boldsymbol{E} \\ \boldsymbol{E}^{\mathrm{T}} & \boldsymbol{C} \end{bmatrix} \begin{bmatrix} \Delta \boldsymbol{x}_c \\ \Delta \boldsymbol{x}_p \end{bmatrix} = \begin{bmatrix} \boldsymbol{I} & -\boldsymbol{E}\boldsymbol{C}^{-1} \\ \boldsymbol{0} & \boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{v} \\ \boldsymbol{w} \end{bmatrix}. \Rightarrow \begin{bmatrix} \boldsymbol{B} - \boldsymbol{E}\boldsymbol{C}^{-1}\boldsymbol{E}^{\mathrm{T}} & \boldsymbol{0} \\ \boldsymbol{E}^{\mathrm{T}} & \boldsymbol{C} \end{bmatrix} \begin{bmatrix} \Delta \boldsymbol{x}_c \\ \Delta \boldsymbol{x}_p \end{bmatrix} = \begin{bmatrix} \boldsymbol{v} - \boldsymbol{E}\boldsymbol{C}^{-1}\boldsymbol{w} \\ \boldsymbol{w} \end{bmatrix}.$$

2. Batch Optimization

- So the normal equation becomes:

$$\begin{bmatrix} \boldsymbol{B} - \boldsymbol{E}\boldsymbol{C}^{-1}\boldsymbol{E}^{\mathrm{T}} & \mathbf{0} \\ \boldsymbol{E}^{\mathrm{T}} & \boldsymbol{C} \end{bmatrix} \begin{bmatrix} \Delta \boldsymbol{x}_c \\ \Delta \boldsymbol{x}_p \end{bmatrix} = \begin{bmatrix} \boldsymbol{v} - \boldsymbol{E}\boldsymbol{C}^{-1}\boldsymbol{w} \\ \boldsymbol{w} \end{bmatrix}.$$

- Solve it in two steps:
  1. Solve the upper part to get Δx_c
  2. Take it into the lower part and get Δx_p
- This is called Marginalization or Schur complement
  - We can also use other approaches like Cholesky decomposition to solve this sparse linear problem

2. Batch Optimization

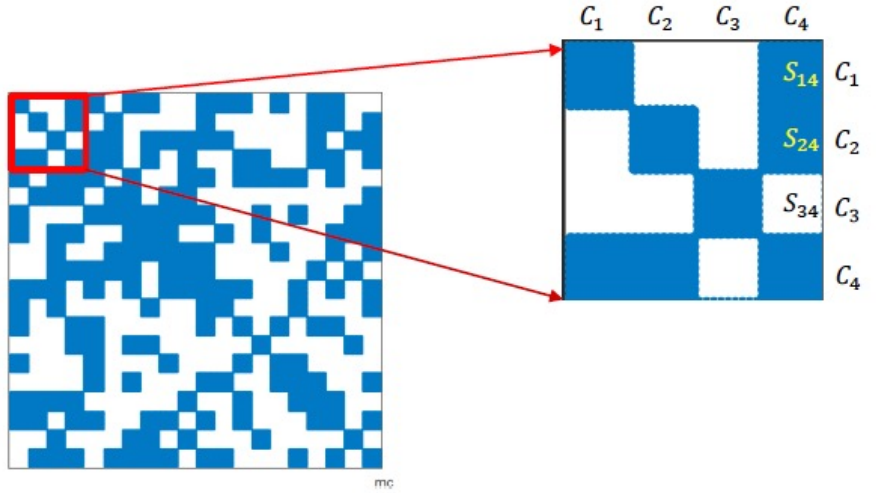
- Marginalization
  - From the probabilistic theory, it means:
  - $P(x_c, x_p) = P(x_c|x_p)P(x_p),$ Joint = Marginal * Conditional

  - In BA, we marginalize all the points into the cameras to make the acceleration
  - And in KF & EKF, we actually marginalize all the past state into the current state

  - We can also choose to marginalize part of the points or part of the cameras

2. Batch Optimization

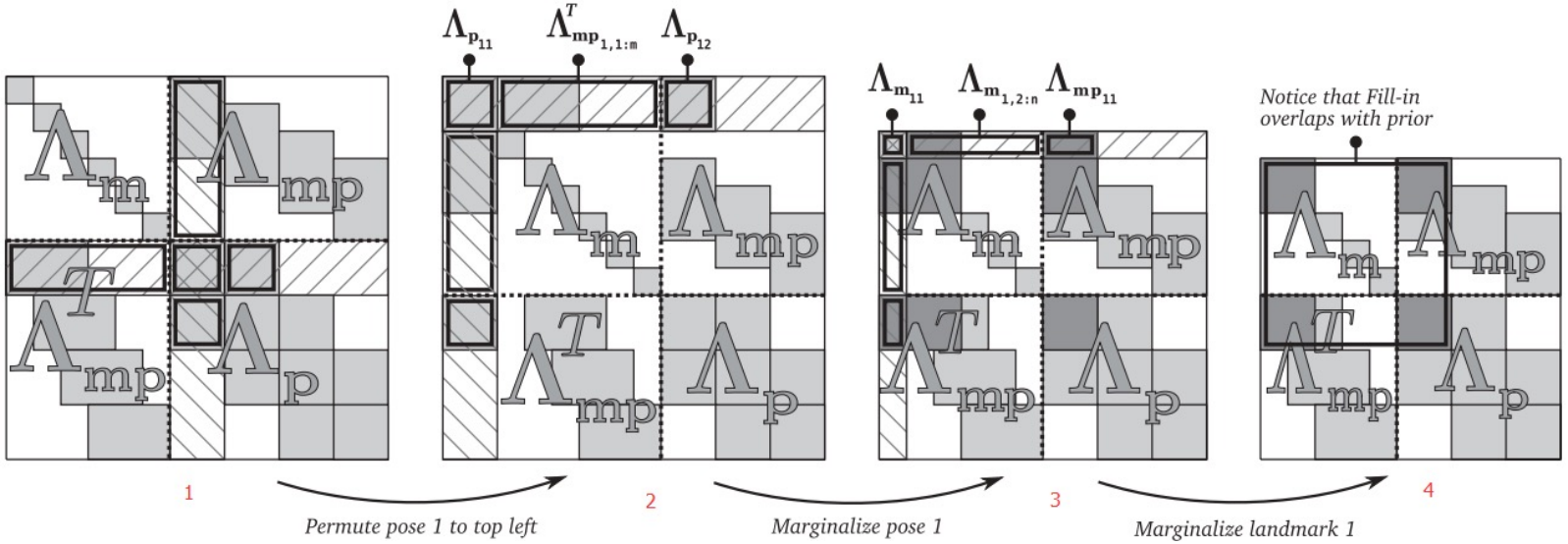
- After marginalization, the top-left corner of H won’ t have the sparse structure again:



- But it shows the co-visibility relationship of the cameras
  - The non-zero block in i,j means camera i and camera j have observed at least one same point

2. Batch Optimization

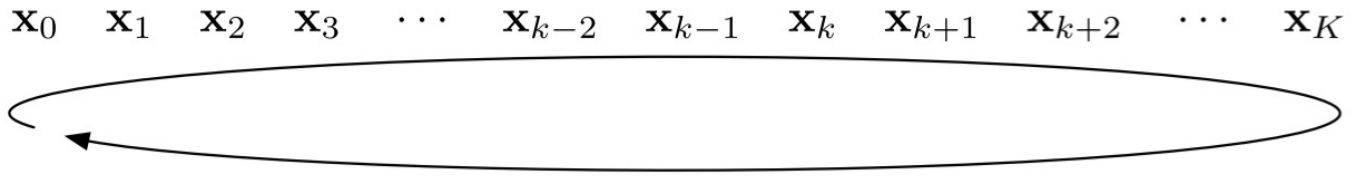
- Marginalization will fill the original matrix and make it no longer sparse
- So in KF & EKF, the covariance matrix is not sparse
- And in recursive problems, we can
  - Just use a dense matrix but keep it small (like EKF, only keeps the current camera estimation)
  - Or use a special marginalization strategy to keep it sparse



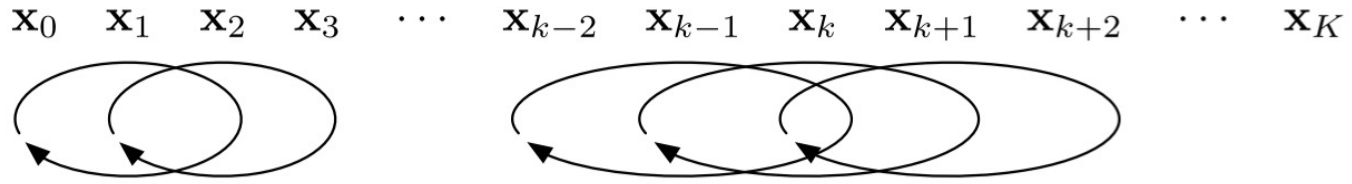
2. Batch Optimization

- Comparison of recursive and batch approaches:

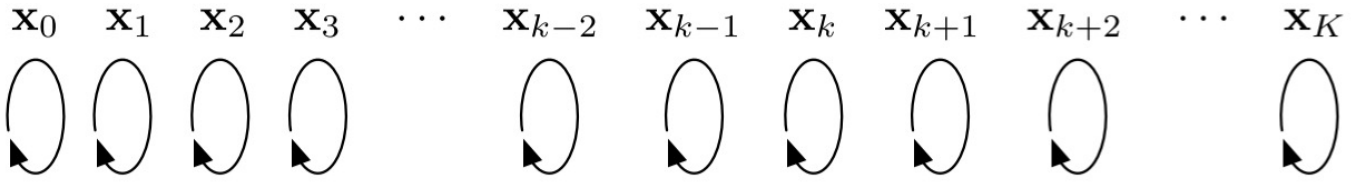
Gauss-Newton iterates over the entire trajectory, but runs offline and not in constant time



Sliding-window filters iterate over several timesteps at once, run online and in constant time

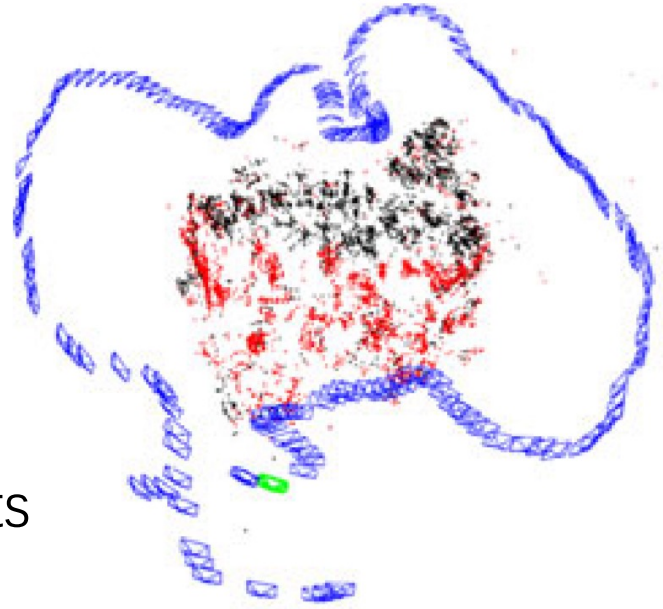


IEKF iterates at only one timestep at a time, but runs online and in constant time



2. Batch Optimization

- Apply BA in SLAM
  - Manage a keyframe set and map point set
- Batch approach
  - Use BA to optimize part of the graph
  - Keep others fixed
- Recursive approach (sliding window)
  - Keep a constant number of keyframes
  - Use BA to optimize the keyframe and points inside the window
  - Marginalize old keyframe and points when new data arrived

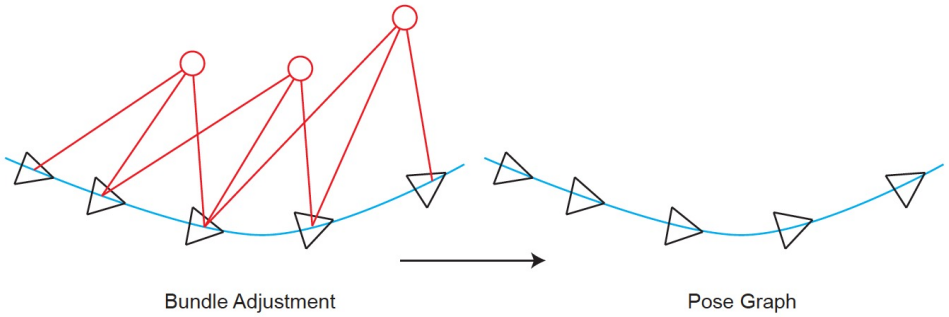


Contents

- Recursive Optimization
- Batch Optimization
- Pose graph

3. Pose graph

- BA usually needs much computation resource
  - So we put it in a single backed thread
  - Modern CPU need several seconds to solve a problem with 100 cameras and 100,000 points
- If we build a problem that only has cameras and no points, then the computation can be greatly reduced



3. Pose graph

- Pose graph
  - Vertex: cameras only
  - Edge: camera transform as observatior $\boldsymbol{\Delta T}_{ij} = \boldsymbol{T}_i^{-1}\boldsymbol{T}_j.$

  - Error: $$\begin{aligned} \boldsymbol{e}_{ij} &= \ln\left(\Delta \boldsymbol{T}_{ij}^{-1}\boldsymbol{T}_i^{-1}\boldsymbol{T}_j\right)^{\vee} \\ &= \ln\left(\exp((-\boldsymbol{\xi}_{ij})^{\wedge})\exp((-\boldsymbol{\xi}_i)^{\wedge})\exp(\boldsymbol{\xi}_j^{\wedge})\right)^{\vee}. \end{aligned}$$

3. Pose graph

- Jacobians

$$
\begin{aligned}
\hat{\boldsymbol{e}}_{ij} &= \ln\left(\boldsymbol{T}_{ij}^{-1}\boldsymbol{T}_i^{-1}\exp((-\boldsymbol{\delta\xi}_i)^\wedge)\exp(\delta\boldsymbol{\xi}_j^\wedge)\boldsymbol{T}_j\right)^\vee \\
&= \ln\left(\boldsymbol{T}_{ij}^{-1}\boldsymbol{T}_i^{-1}\boldsymbol{T}_j\exp\left(\left(-\mathrm{Ad}(\boldsymbol{T}_j^{-1})\boldsymbol{\delta\xi}_i\right)^\wedge\right)\exp\left(\left(\mathrm{Ad}(\boldsymbol{T}_j^{-1})\boldsymbol{\delta\xi}_j\right)^\wedge\right)^\vee \right. \\
&\approx \ln\left(\boldsymbol{T}_{ij}^{-1}\boldsymbol{T}_i^{-1}\boldsymbol{T}_j\left[\boldsymbol{I}-(\mathrm{Ad}(\boldsymbol{T}_j^{-1})\boldsymbol{\delta\xi}_i)^\wedge+(\mathrm{Ad}(\boldsymbol{T}_j^{-1})\boldsymbol{\delta\xi}_j)^\wedge\right]\right)^\vee \\
&\approx \boldsymbol{e}_{ij}+\frac{\partial\boldsymbol{e}_{ij}}{\partial\boldsymbol{\delta\xi}_i}\boldsymbol{\delta\xi}_i+\frac{\partial\boldsymbol{e}_{ij}}{\partial\boldsymbol{\delta\xi}_j}\boldsymbol{\delta\xi}_j
\end{aligned}
$$

$$
\frac{\partial\boldsymbol{e}_{ij}}{\partial\boldsymbol{\delta\xi}_i}=-\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{e}_{ij})\mathrm{Ad}(\boldsymbol{T}_j^{-1}).
$$

$$
\frac{\partial\boldsymbol{e}_{ij}}{\partial\boldsymbol{\delta\xi}_j}=\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{e}_{ij})\mathrm{Ad}(\boldsymbol{T}_j^{-1}).
$$

$$
\boldsymbol{\mathcal{J}}_r^{-1}(\boldsymbol{e}_{ij})\approx\boldsymbol{I}+\frac{1}{2}\begin{bmatrix}\boldsymbol{\phi}_{\boldsymbol{e}}^\wedge & \boldsymbol{\rho}_{\boldsymbol{e}}^\wedge\\ \boldsymbol{0} & \boldsymbol{\phi}_{\boldsymbol{e}}^\wedge\end{bmatrix}.
$$

Adjoint:

$$
\exp\left((\mathrm{Ad}(\boldsymbol{T})\boldsymbol{\xi})^\wedge\right)=\boldsymbol{T}\exp(\boldsymbol{\xi}^\wedge)\boldsymbol{T}^{-1}.
$$

Any Questions?